Training V4 Report: LSTM Sequence Model

Overview

This report documents the results of Phase 3 Version 4. After V3 (Random Forest + SMOTE) failed to generalize (22.22% Test Accuracy) and showed a complete inability to learn time-based errors (0.00 F1-score for late_foot), a new hypothesis was formed:

Hypothesis: The model is failing because the *algorithm* (Random Forest) is wrong. It cannot understand the *sequence* of motion.

Objective: To test this hypothesis by replacing the static Random Forest with a sequence-based **LSTM (Long Short-Term Memory) Neural Network**. The goal is not to achieve perfect accuracy, but to see if this new architecture can learn the minority error classes at all.

Dataset & Pre-processing

- **Samples Loaded:** 224
- **Feature Shape:** (224, 50, 7)
    - This is a critical change. We are no longer flattening data. The model will read the full sequence of 50 frames, each with 7 kinematic features.

Data Preparation

1. **Label Encoding:** String labels (e.g., 'good') were converted to integers (e.g., 0) using LabelEncoder.
2. **Feature Scaling:** The 3D data was unrolled, standardized using StandardScaler to normalize the feature ranges (a requirement for neural networks), and then re-rolled into its original (224, 50, 7) shape.
3. **Train/Test Split:** A stratified split was used.
    - **Training Samples:** 179
    - **Testing Samples:** 45

Model Training: LSTM (V4)

- **Algorithm:** LSTM (Long Short-Term Memory)
    - This is a Recurrent Neural Network (RNN) specifically chosen for its ability to learn patterns over time (sequences).
- **Architecture:** A simple stack was used to prevent overfitting on the small dataset:

- Input(shape=(50, 7))
- LSTM(64, return_sequences=True)
- Dropout(0.3)
- LSTM(32)
- Dropout(0.3)
- Dense(16, 'relu')
- Dense(5, 'softmax')

- **Training:**
  - **Anti-Imbalance:** class_weight='balanced' was used to force the model to pay more attention to the minority error classes.
  - **Anti-Overfitting:** EarlyStopping was used, which monitored validation loss.
  - **Result:** Training automatically stopped at **Epoch 16**, indicating the model learned all it could from the 179 training samples very quickly.

Model Evaluation (V4)

- **Test Accuracy:** 26.67%
- **Test F1-Score (Weighted):** 26.0%

Initial Analysis

The overall accuracy appears low (27%). However, a deep dive into the **Classification Report** reveals the true story.

Classification Report (V4)

Class	**precision**	**recall**	**f1-score**	**support**
good	0.40	0.38	0.39	16
head_falling	0.22	0.25	0.24	8
late_foot	**0.15**	**0.40**	**0.22**	5
low_elbow	**0.20**	**0.20**	**0.20**	5
off_balance	0.33	0.09	0.14	11

macro avg	0.26	0.26	0.24	45
weighted avg	0.30	0.27	0.26	45

Per-Class Notes & Breakthrough

This report is a **major success**:

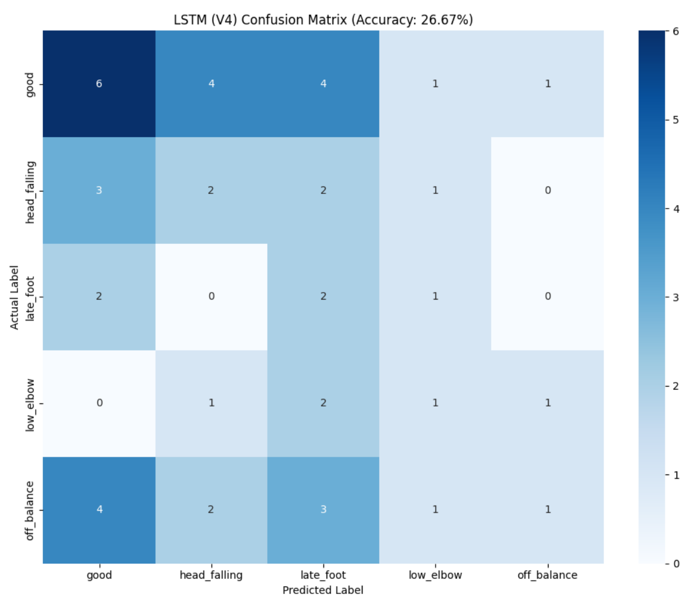
1. **late_foot F1-Score: 0.00 (V3) -> 0.22 (V4)**
2. **low_elbow F1-Score: 0.00 (V3) -> 0.20 (V4)**

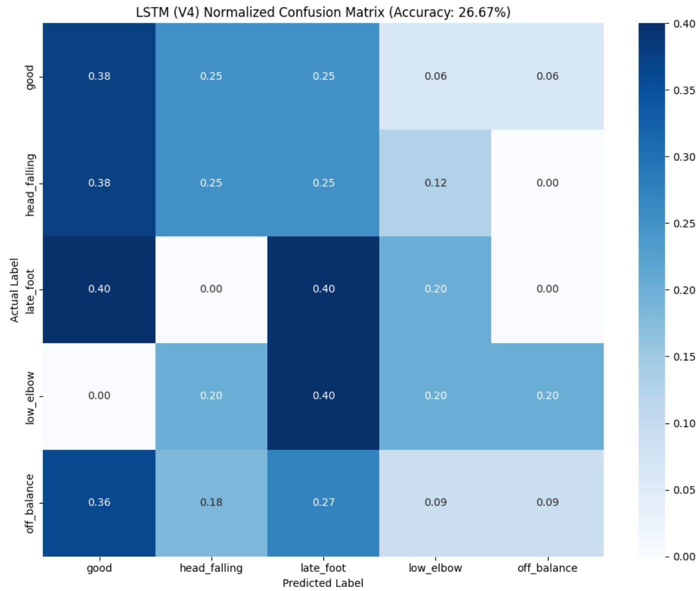
This is the "proof of life" we were looking for. The LSTM model, unlike the Random Forest, is **no longer blind** to the time-based errors.

Furthermore, the **recall** for late_foot is **0.40**, meaning the model *correctly identified 40% (2 out of 5) of the late_foot errors* in the unseen test set.

Visualization:

- **Confusion Matrix:**





- **Training History:**

Final Summary & Conclusion

- **Hypothesis CONFIRMED:** The problem *was* a sequence-based problem. The Random Forest (V3) failed because it could not understand time, while the LSTM (V4) *can*.
- **New Bottleneck:** The model is "data-starved." It stopped training at Epoch 16 because 179 samples is not enough data to effectively train a neural network from scratch. It's "weak" and "under-confident."
- **Overfitting:** The model did not overfit, as the EarlyStopping callback worked correctly. The low accuracy is due to a lack of training data, not memorization.

This concludes Training V4. We have successfully validated our architecture.

Recommended Next Steps

We must address the "data-starved" problem.

1. **Model 1 (Base Model):** Train a large LSTM on the **1,750-video** Hugging Face dataset to learn the *general physics* of cricket shots.
2. **Model 2 (Transfer Learning): Freeze** the "brain" of Model 1, attach a new head, and **fine-tune** it on our high-quality 225 "error" videos. This will transfer the "cricketing knowledge" to our "coaching" model.